

	DIRECCIÓN NACIONAL DE TECNOLOGÍAS DE LA INFORMACIÓN SUBDIRECCIÓN NACIONAL DE ARQUITECTURA Y SOLUCIONES Estándar	COD: RC3 FECHA: 2020-03-13 VERSIÓN: 1.0
---	--	---

Proyecto Base API REST – Spring Boot

Estándar

PAS-EST-055

Versión 2.0

© 2026 Dirección Nacional de Tecnologías de la Información
 TODOS LOS DERECHOS RESERVADOS

Queda reservado el derecho de propiedad de este documento, con la facultad de disponer de él, publicarlo, traducirlo o autorizar su traducción, así como reproducirlo total o parcialmente, por cualquier sistema o medio.

No se permite la reproducción total o parcial de este documento, ni su incorporación a un sistema informático, ni su locación, ni su transmisión en cualquier forma o por cualquier medio, sea éste escrito o electrónico, mecánico, por fotocopia, por grabación u otros métodos, sin el permiso previo y escrito de los titulares de los derechos y del copyright.

FOTOCOPIAR ES DELITO.

Otros nombres de compañías y productos mencionados en este documento pueden ser marcas comerciales o marcas registradas por sus respectivos dueños.

	DIRECCIÓN NACIONAL DE TECNOLOGÍAS DE LA INFORMACIÓN SUBDIRECCIÓN NACIONAL DE ARQUITECTURA Y SOLUCIONES Estándar	COD: RC3 FECHA: 2020-03-13 VERSIÓN: 1.0
---	--	---

FIRMAS Y APROBACIONES			
ELABORADO POR:			
Nombre	Cargo - Unidad	Fecha	Firma
Juan Carlos Estévez	Analista Informático – Subdirección Nacional de Arquitectura y Soluciones	2026-06-30	
REVISADO POR:			
Nombre	Cargo – Unidad	Fecha	Firma
Christian Armijos	Analista Informático – Subdirección Nacional de Desarrollo Informático	2026-06-30	
APROBADO POR:			
Nombre – Cargo	Cargo - Unidad	Fecha	Firma
Andrés Roberto García Moreno	Subdirector Nacional de Arquitectura y Soluciones – Subdirección Nacional de Arquitectura y Soluciones	2026-06-30	

LISTA DE CAMBIOS			
Versión	Fecha	Autor	Descripción
1.0	2026-05-08	Juan Carlos Estévez	Emisión Inicial
2.0	2026-06-30	Juan Carlos Estévez	Actualización del RSA, Pruebas Unitarias

TABLA DE CONTENIDOS

1. ALCANCE	1
2. OBJETIVO.....	1
3. ÁMBITO DE APLICACIÓN	1
4. DEFINICIONES, ABREVIATURAS, ACRÓNIMOS	1
5. ARQUITECTURA DEL PROYECTO	3
5.1 ARQUITECTURA HEXAGONAL	3
5.2 REGLA DE DEPENDENCIAS	3
5.3 DIAGRAMA DE CAPAS	3
6. ESTRUCTURA DE PAQUETES	4
6.1 PAQUETE MODEL	4
6.2 PAQUETE APPLICATION/PORT	4
6.3 PAQUETE APPLICATION/USECASE	4
6.4 PAQUETE INFRASTRUCTURE/CONTROLLER	5
6.5 PAQUETE INFRASTRUCTURE/CONTROLLER/DTO	5
6.6 PAQUETE INFRASTRUCTURE/MAPPER	5
6.7 PAQUETE INFRASTRUCTURE/PERSISTENCE/JPA	5
6.8 PAQUETE INFRASTRUCTURE/PERSISTENCE/MONGO.....	6
6.9 PAQUETE INFRASTRUCTURE/CONFIG	6
7. MODELOS GENÉRICOS.....	6
7.1 MODELO DE DOMINIO	6
7.2 CAMPOS DE AUDITORÍA OBLIGATORIOS	7
7.3 ENTIDAD JPA.....	7
7.4 REPOSITORIO JPA.....	8
8. CASOS DE USO	8
8.1 ESTRUCTURA GENÉRICA	8
8.2 OPERACIONES ESTÁNDAR	9
8.3 MANEJO DE EXCEPCIONES	9
9. DTOS — OBJETOS DE TRANSFERENCIA DE DATOS	9
9.1 REQUEST	9
9.2 RESPONSE	10
9.3 PAGERESPONSE.....	10
10. MAPPERS	10
11. ESTÁNDAR DE COMENTARIOS	11
11.1 CABECERA DE COPYRIGHT	11
11.2 JAVADOC DE CLASE.....	11
11.3 JAVADOC DE MÉTODOS	11
11.4 COMENTARIOS DE CAMPOS.....	11
11.5 COMENTARIOS DE SECCIÓN	12
12. DOCUMENTACIÓN DE ENDPOINTS	12
12.1 CONVENCION DE RUTAS	12
12.2 MÉTODOS HTTP Y OPERACIONES	12
12.3 PARÁMETROS DE PAGINACIÓN	12
12.4 CÓDIGOS HTTP ESTÁNDAR.....	13
12.5 DOCUMENTACIÓN SWAGGER.....	13
13. CONFIGURACIÓN DEL ENTORNO	13

	DIRECCIÓN NACIONAL DE TECNOLOGÍAS DE LA INFORMACIÓN SUBDIRECCIÓN NACIONAL DE ARQUITECTURA Y SOLUCIONES Estándar	COD: RC3 FECHA: 2020-03-13 VERSIÓN: 1.0
---	--	--

13.1 ARCHIVO .ENV	13
13.2 VARIABLES POR CATEGORÍA	13
13.3 PERFILES DE SPRING	14
14. VAULT — GESTIÓN DE SECRETOS.....	14
14.1 ESTRUCTURA DE PATHS	14
14.2 VAULTENVIRONMENTPOSTPROCESSOR	15
14.3 CARGA AUTOMÁTICA DE SECRETOS — VAULT-INIT.SH	15
14.4 AGREGAR UN NUEVO SECRETO	15
14.5 CIFRADO RSA PARA CAMPOS SENSIBLES	15
15. ECOSISTEMA DE SERVICIOS.....	17
15.1 DOCKER-COMPOSE-UTILITARIOS.YML	17
15.2 RED DOCKER	18
15.3 VAULT-INIT.SH	18
16. DESPLIEGUE.....	18
16.1 DESPLIEGUE LOCAL	18
16.2 DESPLIEGUE DOCKERIZADO	18
16.3 DOCKERFILE	19
16.4 VARIABLE IP_SERVER	19
17. AUDITORÍA MONGODB	19
17.1 CONFIGURACIÓN	19
17.2 OPERACIONES CAPTURADAS AUTOMÁTICAMENTE	20
17.3 CAMPOS DEL DOCUMENTO DE AUDITORÍA	20
17.4 EXTENDER LA AUDITORÍA A NUEVAS ENTIDADES	20
17.5 BASE DE DATOS DE AUDITORÍA	21
18. ALMACENAMIENTO DE ARCHIVOS.....	21
18.1 MODOS DE ALMACENAMIENTO	21
18.2 PATRÓN DE ALMACENAMIENTO	21
18.3 BUCKET MINIO	21
18.4 USO DEL STORAGEPORT EN NUEVOS CASOS DE USO	21
19. MEJORES PRÁCTICAS	22
19.1 QUÉ MODIFICAR PARA NUEVA LÓGICA DE NEGOCIO	22
19.2 QUÉ NO MODIFICAR POR LÓGICA DE NEGOCIO	22
19.3 CHECKLIST PARA AGREGAR UNA NUEVA ENTIDAD	22
19.4 REGLAS DE ORO	22
20. PRUEBAS	23
20.1 ESTRUCTURA DE PAQUETES DE PRUEBA	23
20.2 TECNOLOGÍAS Y DEPENDENCIAS	23
20.3 PATRONES DE PRUEBA POR CAPA	23
20.3.1 Pruebas de Casos de Uso (UseCase)	23
20.3.2 Pruebas de Mappers (Mapper)	25
20.3.3 Prueba de Integración	26
20.4 CONVENCIONES DE NOMENCLATURA	26
20.5 COBERTURA ESPERADA POR MÉTODO	27
20.5.1 Casos de Uso	27
20.5.2 Mappers	27
20.6 CHECKLIST PARA AGREGAR PRUEBAS DE NUEVA ENTIDAD	27
21. ANEXOS.....	28
21.1 ANEXO 1 — .ENV.EXAMPLE	28
21.2 ANEXO 2 — DDL GENÉRICO CON CAMPOS DE AUDITORÍA	29
21.3 ANEXO 3 — LOG DE ARRANQUE	29

Estándar

Proyecto Base API REST – Spring Boot

1. Alcance

El presente documento define el estándar de arquitectura, desarrollo, configuración y despliegue del Proyecto Base API REST construido sobre Spring Boot 3, denominado BaseSpringApi. Establece los lineamientos obligatorios que deben seguir todos los equipos de desarrollo de la Dirección Nacional de Tecnologías de la Información del IESS al crear nuevos microservicios o servicios REST basados en esta plantilla.

2. Objetivo

Establecer el estándar de implementación para el Proyecto Base API REST del IESS, definiendo la arquitectura hexagonal, la estructura de paquetes, los patrones de codificación, la gestión de secretos con Vault, la auditoría con MongoDB, el almacenamiento de archivos, el despliegue con Docker y las convenciones de documentación que garanticen consistencia, mantenibilidad y trazabilidad en todos los servicios del ecosistema tecnológico institucional.

3. Ámbito de Aplicación

El presente documento es de aplicación obligatoria para:

- La Dirección Nacional de Tecnología de la Información y todas las unidades informáticas de las dependencias a nivel nacional.
- Todo equipo de desarrollo que implemente un nuevo servicio REST basado en el Proyecto Base API REST del IESS.
- Proyectos nuevos y migraciones que adopten la arquitectura hexagonal con Spring Boot 3.

4. Definiciones, Abreviaturas, Acrónimos

Término	Descripción
Arquitectura Hexagonal	Patrón de arquitectura de software que separa la lógica de negocio del núcleo (dominio) de los adaptadores de entrada y salida mediante puertos e interfaces.
AOP	Aspect Oriented Programming. Programación Orientada a Aspectos. Permite interceptar métodos para ejecutar lógica transversal como auditoría.
DTO	Data Transfer Object. Objeto de transferencia de datos usado para comunicar capas sin exponer el modelo de dominio.
JPA	Jakarta Persistence API. Estándar de persistencia para mapeo objeto-relacional en Java.
JWT	JSON Web Token. Estándar para transmisión segura de información entre partes como objeto JSON firmado.

Término	Descripción
KV	Key-Value. Motor de almacenamiento de secretos de HashiCorp Vault en pares clave-valor.
MinIO	Servidor de almacenamiento de objetos compatible con S3 utilizado para archivos PDF y binarios.
Vault	HashiCorp Vault. Herramienta de gestión centralizada de secretos y credenciales.
IESS	Instituto Ecuatoriano de Seguridad Social.
DNTSI	Dirección Nacional de Tecnologías de la Información.
DDL	Data Definition Language. Lenguaje de definición de datos para bases de datos.
Status A/I/E	Estados lógicos de registro: A=Activo, I=Inactivo, E=Eliminado lógicamente.
UseCase	Caso de uso. Clase que encapsula la lógica de negocio de una operación específica.
Port	Puerto. Interfaz que define el contrato entre la capa de aplicación y la infraestructura.
EnvironmentPostProcessor	Mecanismo de Spring Boot para inyectar propiedades en la fase más temprana del arranque.

5. Arquitectura del Proyecto

5.1 Arquitectura Hexagonal

El proyecto implementa una arquitectura hexagonal simplificada dividida en tres anillos concéntricos:

Núcleo (model)

Contiene las entidades de dominio puras. No tiene dependencia de ningún framework, base de datos ni biblioteca externa. Es la capa más protegida del sistema.

Aplicación (application)

Contiene los casos de uso (usecase) que orquestan la lógica de negocio y los puertos (port) que definen los contratos de comunicación con la infraestructura. Solo depende del núcleo.

Infraestructura (infrastructure)

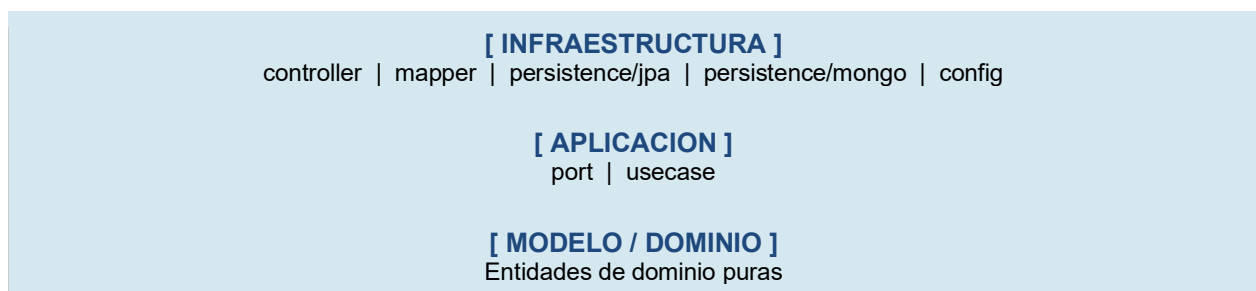
Contiene todos los adaptadores: controllers REST, entidades JPA, repositorios, mappers y configuraciones. Es la única capa que conoce frameworks y tecnologías externas.

5.2 Regla de Dependencias

Las dependencias siempre apuntan hacia adentro. Nunca hacia afuera:

- infrastructure → application → model
- El model no conoce a nadie.
- Los controllers llaman solo a interfaces de port, nunca directamente a repositorios.
- Los casos de uso llaman a repositorios JPA directamente (simplificación adoptada).

5.3 Diagrama de Capas



6. Estructura de Paquetes

El paquete raíz del proyecto sigue la convención: {groupId}.{artifactId}

Ejemplo: iess.gen.BaseSpringApi

6.1 Paquete model

Elemento	Descripción
Ubicación	iess.{modulo}.{servicio}.model
Contenido	Clases POJO de dominio. Sin anotaciones de Spring, JPA ni frameworks externos.
Convención de nombre	{Entidad}.java — Ejemplo: Persona.java, Documento.java
Restricción	No debe importar nada de infrastructure ni de application.

6.2 Paquete application/port

Elemento	Descripción
Ubicación	iess.{modulo}.{servicio}.application.port
Contenido	Interfaces que definen contratos entre la aplicación y la infraestructura.
Convención de nombre	{Entidad}RepositoryPort.java, StoragePort.java
Restricción	Solo interfaces. Cero implementaciones. Solo depende de model.

6.3 Paquete application/usecase

Elemento	Descripción
Ubicación	iess.{modulo}.{servicio}.application.usecase
Contenido	Clases @Service con la lógica de negocio. Un UseCase por entidad principal.
Convención de nombre	{Entidad}UseCase.java — Ejemplo: PersonaUseCase.java
Métodos estándar	crear(), actualizar(), cambiarStatus(), eliminar(), buscarPorId(), listarActivos()
Restricción	No debe tener lógica HTTP ni conocer request/response. Solo lógica de negocio.

6.4 Paquete infrastructure/controller

Elemento	Descripción
Ubicación	iess.{modulo}.{servicio}.infrastructure.controller
Contenido	@RestController que recibe peticiones HTTP y delega al usecase.
Convención de nombre	{Entidad}Controller.java
Restricción	No debe tener lógica de negocio. Solo validar, delegar y responder.

6.5 Paquete infrastructure/controller/dto

Elemento	Descripción
Ubicación	iess.{modulo}.{servicio}.infrastructure.controller.dto
Contenido	DTOs de entrada (Request) y salida (Response). PageResponse genérico.
Clases obligatorias	{Entidad}Request.java, {Entidad}Response.java, PageResponse.java
Restricción	Request no expone campos de auditoría. Response sí los incluye.

6.6 Paquete infrastructure/mapper

Elemento	Descripción
Ubicación	iess.{modulo}.{servicio}.infrastructure.mapper
Contenido	@Component con las 4 conversiones por entidad.
Convención de nombre	{Entidad}Mapper.java
Métodos obligatorios	toDomain(Entity), toEntity(Domain), toDomain(Request), toResponse(Domain)
Restricción	Sin lógica de negocio. Solo conversión de tipos.

6.7 Paquete infrastructure/persistence/jpa

Elemento	Descripción
Ubicación	iess.{modulo}.{servicio}.infrastructure.persistence.jpa
Contenido	Entidades @Entity y repositorios JpaRepository de Spring Data.
Convención de nombre	{Entidad}Entity.java, {Entidad}JpaRepository.java
Restricción	Las entidades JPA nunca salen de este paquete hacia capas superiores.

6.8 Paquete infrastructure/persistence/mongo

Elemento	Descripción
Ubicación	iess.{modulo}.{servicio}.infrastructure.persistence.mongo
Contenido	Documento @Document para auditoría y su repositorio MongoRepository.
Clases	AuditoriaDocument.java, AuditoriaMongoRepository.java
Uso exclusivo	Auditoría de operaciones. No almacena datos de negocio transaccional.

6.9 Paquete infrastructure/config

Elemento	Descripción
Ubicación	iess.{modulo}.{servicio}.infrastructure.config
Contenido	Clases @Configuration para beans de Spring y servicios externos.
Clases presentes	JpaAuditingConfig, MongoConfig, MinioConfig, SwaggerConfig, VaultEnvironmentPostProcessor, AuditoriaMongoAspect, ApplicationStartupLogger, RsaConfig
Restricción	Solo @Configuration, @Bean y @EnableXxx. Cero lógica de negocio.

7. Modelos Genéricos

7.1 Modelo de Dominio

Todo modelo de dominio debe seguir la siguiente estructura genérica:

```
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class {Entidad} {
    // Identificador
    private UUID id;

    // Atributos de negocio propios de la entidad
    private String campo1;
    private String campo2;

    // — Auditoría obligatoria —————
    private String status; // A=Activo, I=Inactivo, E=Eliminado
    private String createdBy;
    private LocalDateTime createdAt;
    private String updatedBy;
    private LocalDateTime updatedAt;
    private String deletedBy; // null si está activo
    private LocalDateTime deletedAt; // null si está activo
}
```

7.2 Campos de Auditoría Obligatorios

Toda entidad del sistema debe incluir los siguientes 7 campos de auditoría:

Campo	Tipo Java	Columna BD	Descripción
status	String	status CHAR(1)	A=Activo, I=Inactivo, E=Eliminado lógico. Nunca borrado físico.
createdBy	String	created_by VARCHAR(100)	Usuario de aplicación que creó el registro.
createdAt	LocalDateTime	created_at TIMESTAMP	Fecha y hora exacta de creación. No actualizable.
updatedBy	String	updated_by VARCHAR(100)	Usuario que realizó la última modificación.
updatedAt	LocalDateTime	updated_at TIMESTAMP	Fecha y hora de última modificación.
deletedBy	String	deleted_by VARCHAR(100)	Usuario que realizó la eliminación lógica. NULL si activo.
deletedAt	LocalDateTime	deleted_at TIMESTAMP	Fecha de eliminación lógica. NULL si activo.

Los campos createdBy, createdAt, updatedBy y updatedAt son gestionados automáticamente por Spring Data JPA Auditing mediante las anotaciones @CreatedBy, @CreatedDate, @LastModifiedBy y @LastModifiedDate en la entidad JPA.

7.3 Entidad JPA

Toda entidad JPA debe seguir la siguiente estructura:

```
@Entity
@Table(name = "{nombre_tabla}",
    indexes = {
        @Index(name = "idx_{tabla}_status", columnList = "status")
    })
@EntityListeners(AuditingEntityListener.class)
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class {Entidad}Entity {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    @Column(name = "id", updatable = false, nullable = false)
    private UUID id;

    // Atributos de negocio con @Column

    @Column(name = "status", nullable = false, length = 1)
    @Builder.Default
    private String status = "A";

    @CreatedBy @Column(name = "created_by", updatable = false, length = 100)
```

```

private String createdBy;

@CreatedDate @Column(name = "created_at", nullable = false, updatable = false)
private LocalDateTime createdAt;

@LastModifiedBy @Column(name = "updated_by", length = 100)
private String updatedBy;

@LastModifiedDate @Column(name = "updated_at")
private LocalDateTime updatedAt;

@Column(name = "deleted_by", length = 100)
private String deletedBy;

@Column(name = "deleted_at")
private LocalDateTime deletedAt;
}

```

7.4 Repositorio JPA

Todo repositorio debe extender `JpaRepository` e incluir como mínimo:

```

@Repository
public interface {Entidad}JpaRepository extends JpaRepository<{Entidad}Entity,
UUID> {

    Optional<{Entidad}Entity> findByIdAndStatus(UUID id, String status);
    Page<{Entidad}Entity> findAllByStatus(String status, Pageable pageable);
}

```

8. Casos de Uso

8.1 Estructura Genérica

Todo caso de uso debe seguir esta estructura:

```

@Service @RequiredArgsConstructor
public class {Entidad}UseCase {

    private final {Entidad}JpaRepository repository;
    private final {Entidad}Mapper mapper;

    @Transactional
    public {Entidad} crear({Entidad} entidad) { ... }

    @Transactional
    public {Entidad} actualizar(UUID id, {Entidad} datos) { ... }

    @Transactional
    public void cambiarStatus(UUID id, String status) { ... }
}

```

```

@Transactional
public void eliminar(UUID id, String deletedBy) { ... }

@Transactional(readOnly = true)
public {Entidad} buscarPorId(UUID id) { ... }

@Transactional(readOnly = true)
public Page<{Entidad}> listarActivos(Pageable pageable) { ... }
}

```

8.2 Operaciones Estándar

Método	Status requerido	Descripción
crear()	Ninguno — asigna A	Valida unicidad, asigna status=A y persiste.
actualizar()	A	Busca por id y status=A, actualiza campos de negocio.
cambiarStatus()	Cualquiera	Cambia status entre A e I. No acepta E.
eliminar()	A	Asigna status=E, deleted_by y deleted_at. Nunca borra físicamente.
buscarPorId()	A	Busca solo registros activos. Lanza excepción si no existe.
listarActivos()	A	Retorna Page<Entidad> con todos los registros activos.

8.3 Manejo de Excepciones

Los casos de uso usan `IllegalArgumentException` para errores de negocio. El `GlobalExceptionHandler` los intercepta y retorna HTTP 400 con estructura JSON estándar:

```

{
  "timestamp": "2026-05-08T10:00:00",
  "status": 400,
  "error": "Bad Request",
  "mensaje": "Ya existe un registro con la cédula: 1234567890"
}

```

9. DTOs — Objetos de Transferencia de Datos

9.1 Request

El DTO de request contiene únicamente los campos que el cliente puede enviar. Nunca incluye id, campos de auditoría ni status.

```

@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class {Entidad}Request {
    @NotBlank(message = "El campo es obligatorio")
    private String campol;
}

```

```

        // Solo campos de negocio con sus validaciones @Valid
    }
}

```

9.2 Response

El DTO de response incluye todos los campos del negocio más los 7 campos de auditoría. Nunca expone contraseñas ni datos sensibles.

```

@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class {Entidad}Response {
    private UUID id;
    private String campol;
    // Campos de negocio...

    // Auditoría
    private String status;
    private String createdBy;
    private LocalDateTime createdAt;
    private String updatedBy;
    private LocalDateTime updatedAt;
    private String deletedBy;
    private LocalDateTime deletedAt;
}

```

9.3 PageResponse

Respuesta genérica para todos los endpoints de listado paginado:

```

PageResponse<{Entidad}Response> {
    List<{Entidad}Response> contenido;
    int paginaActual;
    int tamanoPagina;
    long totalElementos;
    int totalPaginas;
    boolean primera;
    boolean ultima;
}

```

10. Mappers

Cada entidad tiene un único mapper centralizado con las 4 conversiones obligatorias:

Método	Entrada	Salida	Cuándo usar
toDomain(Entity)	{Entidad}Entity	{Entidad}	Al leer de la BD para retornar al caso de uso.
toEntity(Domain)	{Entidad}	{Entidad}Entity	Al guardar en la BD desde el caso de uso.
toDomain(Request)	{Entidad}Request	{Entidad}	Al recibir datos del cliente en el controller.

Método	Entrada	Salida	Cuándo usar
toResponse(Domain)	{Entidad}	{Entidad}Response	Al devolver datos al cliente desde el controller.

El mapper no contiene lógica de negocio. Solo convierte tipos. Para listas, el mapper se aplica con `.stream().map(mapper::toDomain).toList()`.

11. Estándar de Comentarios

11.1 Cabecera de Copyright

Todo archivo Java debe comenzar con la cabecera de copyright institucional:

```
/**
 * Copyright 2026 INSTITUTO ECUATORIANO DE SEGURIDAD SOCIAL - ECUADOR.
 * Todos los derechos reservados.
 */
```

11.2 Javadoc de Clase

Toda clase pública debe tener Javadoc con la siguiente estructura:

```
/**
 * <b> Descripción de la responsabilidad de la clase. </b>
 *
 * @author Juan Carlos Estévez Hidalgo
 *
 * @version Revision: 1.0
 *
 * <p>
 * [Author: Juan Carlos Estévez Hidalgo , Date: 07 may 2026]
 * </p>
 */
```

11.3 Javadoc de Métodos

Todo método público debe documentarse con:

```
/**
 * <b> Descripción de lo que hace el método. </b>
 *
 * @param nombreParam descripción del parámetro
 * @return descripción del valor retornado
 * @throws IllegalArgumentException si ocurre la condición X
 */
```

11.4 Comentarios de Campos

Los campos de clase deben documentarse con comentario de línea simple:

```
/** Descripción del campo. */  
private String campo;
```

11.5 Comentarios de Sección

Para separar grupos lógicos de campos en una clase se usa:

```
// — Auditoría —————
```

12. Documentación de Endpoints

12.1 Convención de Rutas

Todos los endpoints siguen la convención REST estándar bajo el context-path /api:

```
/api/{entidad-en-plural}  
/api/{entidad-en-plural}/{id}  
/api/{entidad-en-plural}/{id}/status
```

No se incluye versión en la ruta (sin /v1/, /v2/).

12.2 Métodos HTTP y Operaciones

Método HTTP	Operación	Ruta	Código éxito
POST	Crear	/api/{entidad}	201 Created
PUT	Actualizar	/api/{entidad}/{id}	200 OK
GET	Buscar por ID	/api/{entidad}/{id}	200 OK
GET	Listar paginado	/api/{entidad}	200 OK
PATCH	Cambiar status	/api/{entidad}/{id}/status?status=l	204 No Content
DELETE	Eliminación lógica	/api/{entidad}/{id}?deletedBy=system	204 No Content

12.3 Parámetros de Paginación

Parámetro	Tipo	Default	Descripción
pagina	int	0	Número de página base 0.
tamano	int	10	Cantidad de registros por página.
ordenarPor	String	Varía por entidad	Campo por el que se ordena.
direccion	String	asc	Dirección: asc o desc.

12.4 Códigos HTTP Estándar

Código	Significado	Cuándo se usa
200 OK	Éxito	GET, PUT con respuesta de datos.
201 Created	Creado	POST exitoso.
204 No Content	Sin contenido	PATCH y DELETE exitosos.
400 Bad Request	Error del cliente	Validación fallida o regla de negocio violada.
500 Internal Server Error	Error del servidor	Excepción no controlada.

12.5 Documentación Swagger

Todo controller debe anotarse con @Tag y cada endpoint con @Operation:

```
@Tag(name = "{Entidad}s", description = "Gestión de {entidad}s")
@Operation(summary = "Descripción breve del endpoint")
```

Swagger UI disponible en: <http://{host}:{puerto}/api/swagger-ui.html>

13. Configuración del Entorno

13.1 Archivo .env

El proyecto usa spring-dotenv para cargar automáticamente el archivo .env al arrancar. El archivo .env NUNCA debe subirse al repositorio Git. Se versiona solo el .env.example como plantilla.

13.2 Variables por Categoría

Variable	Descripción	¿Gestionada por Vault?
SERVER_PORT	Puerto del servidor HTTP	No
SERVER_CONTEXT_PATH	Prefijo de todas las rutas	No
IP_SERVER	Vacío=local, IP=dockerizado	No
DB_ENGINE	Motor relacional: postgres u oracle	No
DB_POSTGRES_HOST/PORT/NAME/USER/PASS	Conexión a PostgreSQL	Sí
DB_ORACLE_HOST/PORT/SERVICE/USER/PASS	Conexión a Oracle 19c	Sí
MONGO_ENABLED	true activa auditoría MongoDB	No
DB_MONGO_HOST/PORT/NAME/USER/PASS/AUTH	Conexión a MongoDB	Sí
VAULT_ENABLED	true activa lectura de secretos desde Vault	No

Variable	Descripción	¿Gestionada por Vault?
VAULT_HOST/PORT/SCHEME/TOKEN	Conexión a Vault	No
MINIO_ENABLED	true activa almacenamiento en MinIO	No
MINIO_URL/ACCESS_KEY/SECRET_KEY	Conexión a MinIO	Sí
DB_POSTGRES_DDL_AUTO	update en desarrollo, none en producción	No
SWAGGER_ENABLED	true habilita la documentación Swagger	No
RSA_ENABLED	true activa el descifrado RSA de campos sensibles	No
RSA_PUBLIC_KEY	Clave pública RSA en formato X.509 DER (Base64)	Si
RSA_PRIVATE_KEY	Clave privada RSA en formato PKCS8 DER (Base64)	Si

13.3 Perfiles de Spring

El motor de base de datos activo se controla con DB_ENGINE. Spring activa el perfil correspondiente en application.yaml:

DB_ENGINE	Perfil activo	Motor
postgres	postgres	PostgreSQL 16 — desarrollo local Docker
oracle	oracle	Oracle 19c — motor institucional externo

14. Vault — Gestión de Secretos

14.1 Estructura de Paths

Los secretos se organizan en Vault bajo el motor KV v2 habilitado en el path BaseSpringApi/:

Path en Vault	Claves	Variables del sistema
BaseSpringApi/database/postgres	host, port, username, password, bdd	DB_POSTGRES_*
BaseSpringApi/database/oracle	host, port, username, password, service	DB_ORACLE_*
BaseSpringApi/database/mongo	host, port, bdd, username, password, auth_db	DB_MONGO_*
BaseSpringApi/storage/minio	url, accessKey, secretKey	MINIO_*

Path en Vault	Claves	Variables del sistema
BaseSpringApi/seguridad/rsa	publicKey, privateKey	RSA_PUBLIC_KEY, RSA_PRIVATE_KEY

14.2 VaultEnvironmentPostProcessor

La clase VaultEnvironmentPostProcessor implementa EnvironmentPostProcessor y se registra en META-INF/spring.factories. Se ejecuta en la fase más temprana del arranque de Spring Boot, antes de que cualquier bean sea construido, garantizando que el datasource reciba las credenciales correctas desde Vault.

14.3 Carga Automática de Secretos — vault-init.sh

El script vault-init.sh se ejecuta automáticamente al levantar el ecosistema de utilitarios. Lee la variable IP_SERVER del entorno:

- IP_SERVER vacío o localhost → carga todos los hosts como localhost (desarrollo local).
- IP_SERVER=192.168.x.x → reemplaza todos los hosts con esa IP (aplicativo dockerizado).

14.4 Agregar un Nuevo Secreto

Para agregar un nuevo secreto al sistema se deben modificar 3 archivos:

1. vault-init.sh — agregar el bloque vault kv put con las claves del nuevo servicio.
2. VaultEnvironmentPostProcessor.java — agregar la llamada a leerSecreto() con el mapeo de claves a variables del sistema.
3. .env y .env.example — agregar las variables como fallback cuando VAULT_ENABLED=false.

14.5. Cifrado RSA para Campos Sensibles

Propiedad	Valor
Algoritmo	RSA/ECB/OAEPWithSHA-256AndMGF1Padding
Longitud de clave	2048 bits
Modo de cifrado	OAEP (Optimal Asymmetric Encryption Padding)
Hash OAEP	SHA-256
Hash MGF1	SHA-256
Label	null

Formato de clave pública	X.509 / DER (Base64)
Formato de clave privada	PKCS8 / DER (Base64)
Codificación del cifrado	Base64

Nota: El nombre "ECB" es una convención interna del provider SunJCE de Java; RSA no opera por bloques.

Feature Flag

La propiedad `RSA_ENABLED` controla globalmente si el cifrado RSA está activo:

RSA_ENABLED	Efecto
true	Se crean los beans <code>RsaConfig</code> y <code>RsaCipherUtil</code> . Los campos marcados se descifran con RSA. Si el valor no está cifrado correctamente, el endpoint responde 400 Bad Request.
false	No existe <code>RsaCipherUtil</code> . Los campos se almacenan tal como llegan (texto plano).

Endpoint de Clave Pública

GET `/v1/seguridad/rsa/public-key`

Respuesta:

JSON

```
{
  "algoritmo": "RSA",
  "formato": "X.509",
  "publicKey": "<clave pública en Base64 / DER>"
}
```

El valor `publicKey` se entrega en formato X.509 DER codificado en Base64, sin cabeceras PEM.

Para Clientes Externos

El cliente debe:

1. Obtener la clave pública desde `GET /v1/seguridad/rsa/public-key`.
2. Cifrar el valor de la siguiente manera:

Parámetro	Especificación
------------------	-----------------------

Algoritmo asimétrico	RSA
Modo de cifrado	RSA-OAEPH
Hash OAEP	SHA-256
Función MGF	MGF1 con SHA-256
Codificación de entrada	UTF-8
Codificación de salida	Base64 (estándar, sin caracteres adicionales)
Clave pública	X.509 DER, decodificada desde Base64

Colocar el valor cifrado (Base64) en el campo correspondiente del JSON de la petición.

Comportamiento esperado:

- RSA_ENABLED=true y atributo en texto plano arroja 400 Bad Request.
- RSA_ENABLED=true y atributo cifrado correctamente arroja 200/201, el servidor descifra y almacena el valor en texto plano.
- RSA_ENABLED=false arroja el servidor ignora el cifrado, el valor se almacena tal cual.

Consideraciones

- La clave privada nunca se expone. Solo el backend tiene acceso a RSA_PRIVATE_KEY.
- Rotación de claves: regenerar el par periódicamente y actualizar Vault.
- Tamaño del cifrado: RSA 2048 + OAEP produce ~344 caracteres Base64.

Campos con validaciones estrictas: atributos con @Email, @Pattern o @Size deben considerar que el DTO recibe el valor cifrado; el descifrado debe ocurrir antes de la validación o el campo no debe tener anotaciones que el Base64 no pueda cumplir.

15. Ecosistema de Servicios

15.1 docker-compose-utilitarios.yml

Contiene todos los servicios de infraestructura necesarios para el funcionamiento del proyecto:

Servicio	Imagen	Puerto	Descripción
postgres	postgres:16-alpine	5432	Motor relacional principal para desarrollo local.
mongo	mongo:7-jammy	27017	Base de datos de auditoría AUDITORIA_IESS.
vault	hashicorp/vault:1.15	8200	Servidor de gestión de secretos en modo dev.

Servicio	Imagen	Puerto	Descripción
vault-init	hashicorp/vault:1.15	—	Ejecuta vault-init.sh al arrancar. Se cierra al terminar.
minio	minio/minio:2024-11	9000/9001	Almacenamiento de objetos. Consola web en 9001.

15.2 Red Docker

Todos los servicios comparten la red Docker llamada net de tipo bridge. Esta red debe existir antes de levantar cualquier docker-compose. Si no existe, docker-compose-utilitarios.yml la crea automáticamente.

```
docker-compose -f docker-compose-utilitarios.yml up -d
```

15.3 vault-init.sh

Script de shell que carga los secretos iniciales en Vault. Se encuentra en la raíz del proyecto y es el único archivo que debe modificarse para actualizar credenciales o agregar nuevos secretos. Para reejecutarlo sin reiniciar todo el ecosistema:

```
docker-compose -f docker-compose-utilitarios.yml up vault-init
```

16. Despliegue

16.1 Despliegue Local

El aplicativo corre directamente en el sistema operativo del desarrollador usando cualquier IDE o línea de comandos. Los utilitarios corren en Docker en la misma máquina y son accesibles via localhost.

Requisitos:

- JDK 21 instalado en el sistema.
- Maven 3.x disponible en el PATH.
- Docker Desktop corriendo con el ecosistema de utilitarios levantado.
- Archivo .env configurado con IP_SERVER vacío.

Ejecución:

```
# Desde línea de comandos
mvn spring-boot:run

# Desde IDE
Ejecutar la clase BaseSpringApiApplication.java
```

16.2 Despliegue Dockerizado

El aplicativo corre dentro de un contenedor Docker. En este escenario, localhost dentro del contenedor apunta al propio contenedor y no al host de la máquina. Se requiere indicar la IP real del servidor donde corren los utilitarios mediante la variable IP_SERVER.

Requisitos:

- Docker Desktop corriendo.
- Ecosistema de utilitarios levantado y red net creada.
- Archivo .env con IP_SERVER=<IP real del servidor>.

Ejecución:

```
# Construir y levantar
docker-compose up -d --build

# Solo levantar sin reconstruir
docker-compose up -d

# Ver logs
docker-compose logs -f

# Detener
docker-compose down
```

16.3 Dockerfile

El proyecto usa un multi-stage build para minimizar el tamaño de la imagen final:

Stage	Imagen base	Propósito
builder	eclipse-temurin:21-jdk-alpine	Compila el proyecto con Maven y genera el JAR.
runtime	eclipse-temurin:21-jre-alpine	Imagen final con solo el JRE y el JAR (~200MB).

La imagen final corre con usuario no root (appuser) por seguridad. La JVM está optimizada para contenedores con -XX:+UseContainerSupport y -XX:MaxRAMPercentage=75.0.

16.4 Variable IP_SERVER

Escenario	IP_SERVER	Efecto
Desarrollo local	vacío	Vault y todos los servicios se alcanzan via localhost.
Aplicativo dockerizado	192.168.x.x	vault-init.sh carga los hosts con esa IP. Spring Boot la usa para conectarse.

17. Auditoría MongoDB

17.1 Configuración

La auditoría MongoDB se activa con MONGO_ENABLED=true en el .env. Cuando está activa, el AuditoriaMongoAspect intercepta automáticamente todos los métodos de escritura de los casos de uso y registra el evento en MongoDB.

17.2 Operaciones Capturadas Automáticamente

Método interceptado	Operación	Colección destino
*.crear()	I — INSERT	AUD_GEN.{ENTIDAD}
*.subir()	I — INSERT	AUD_GEN.{ENTIDAD}
*.actualizar()	U — UPDATE	AUD_GEN.{ENTIDAD}
*.eliminar()	D — DELETE LÓGICO	AUD_GEN.{ENTIDAD}
*.cambiarStatus()	D — DELETE LÓGICO	AUD_GEN.{ENTIDAD}

17.3 Campos del Documento de Auditoría

Campo MongoDB	Tipo	Descripción
aud_operation	String	Tipo: I=INSERT, U=UPDATE, D=DELETE lógico.
aud_timestamp	LocalDateTime	Fecha y hora exacta del evento.
aud_db_user	String	Usuario de base de datos. Valor: system.
aud_app_user	String	Usuario de aplicación. Provisto por JWT cuando se integre seguridad.
aud_ip_address	String	IP del cliente. Soporta X-Forwarded-For para proxies.
aud_session_id	String	ID de sesión HTTP para correlacionar eventos.
aud_program	String	Módulo y método que ejecutó la operación.
aud_old_values	Map	Snapshot del registro antes del cambio. NULL en INSERT.
aud_new_values	Map	Snapshot del registro después del cambio. NULL en DELETE.
aud_changed_fields	List<String>	Campos que cambiaron en UPDATE.
aud_entity	String	Nombre de la entidad auditada. Ejemplo: PERSONA.
aud_collection	String	Colección MongoDB. Ejemplo: AUD_GEN.PERSONA.

17.4 Extender la Auditoría a Nuevas Entidades

El AuditoriaMongoAspect intercepta automáticamente todos los métodos crear(), actualizar(), eliminar() y cambiarStatus() de cualquier clase en el paquete application.usecase. Por lo tanto, al agregar un nuevo caso de uso, la auditoría funciona sin modificación adicional.

Para capturar old_values en el método actualizar() de una nueva entidad, agregar el repositorio JPA correspondiente en AuditoriaMongoAspect y extender el método leerEstadoPrevio():

```
if ("{"ENTIDAD}").equals(entidad) && {entidad}JpaRepository != null) {  
    return {entidad}JpaRepository.findById(id)  
        .map(entity -> convertirAMapa({entidad}Mapper.toDomain(entity)))  
        .orElse(null);  
}
```


17.5 Base de Datos de Auditoría

Todos los documentos de auditoría se almacenan en la base de datos AUDITORIA_IESS de MongoDB. Esta base de datos se crea automáticamente al primer registro de auditoría. Para visualizarla usar MongoDB Compass:

```
mongodb://mongo_user:mongo_password@localhost:27017/AUDITORIA_IESS?authSource=admin
```

18. Almacenamiento de Archivos

18.1 Modos de Almacenamiento

Modo	Flag	Implementación	Ubicación
Local	MINIO_ENABLED=false	LocalStorageAdapter	Carpeta uploads/ en el servidor.
MinIO	MINIO_ENABLED=true	MinioStorageAdapter	Bucket base-spring-bucket en MinIO.

El cambio entre modos es transparente para los casos de uso gracias al patrón de puerto StoragePort. Cambiar MINIO_ENABLED en el .env es suficiente para alternar entre modos.

18.2 Patrón de Almacenamiento

La metadata del archivo (nombre, ruta, tipo MIME, tamaño) se persiste en la base de datos relacional. El archivo físico se guarda en el almacenamiento configurado. El nombre de almacenamiento usa UUID para evitar colisiones:

```
{UUID}.{extension} → Ejemplo: 7f3a2b1c-4d5e-6789-abcd-ef0123456789.pdf
```

18.3 Bucket MinIO

El bucket base-spring-bucket se crea automáticamente al arrancar si MINIO_ENABLED=true y el bucket no existe. La consola web de MinIO está disponible en:

```
http://{host}:9001 - Usuario: minioadmin Contraseña: minioadmin
```

18.4 Uso del StoragePort en Nuevos Casos de Uso

Para usar el almacenamiento en un nuevo caso de uso, inyectar el StoragePort por constructor:

```
private final StoragePort storagePort;

// Almacenar
String nombreUnico = UUID.randomUUID() + "." + extension;
String ruta = storagePort.almacenar(archivo, nombreUnico);
String bucket = storagePort.getBucket(); // null si es local
```

19. Mejores Prácticas

19.1 Qué Modificar para Nueva Lógica de Negocio

Archivo	Cuándo modificarlo
application/usecase/{Entidad}UseCase.java	Para agregar, cambiar o quitar lógica de negocio.
model/{Entidad}.java	Para agregar un nuevo atributo de negocio a la entidad.
infrastructure/controller/{Entidad}Controller.java	Para agregar o modificar endpoints HTTP.
infrastructure/controller/dto/{Entidad}Request.java	Para cambiar los campos que el cliente puede enviar.
infrastructure/controller/dto/{Entidad}Response.java	Para cambiar los campos que el sistema retorna.
vault-init.sh	Para agregar o modificar secretos en Vault.
VaultEnvironmentPostProcessor.java	Para mapear nuevos secretos de Vault a propiedades de Spring.

19.2 Qué NO Modificar por Lógica de Negocio

- infrastructure/config/ — Solo para cambios de configuración de infraestructura.
- infrastructure/persistence/jpa/ — Solo si cambia el modelo de datos en BD.
- infrastructure/mapper/ — Solo si cambia el modelo de dominio o los DTOs.
- application/port/ — Solo si se agrega un nuevo adaptador de infraestructura.

19.3 Checklist para Agregar una Nueva Entidad

4. Crear el modelo de dominio en model/{Entidad}.java con los 7 campos de auditoría.
5. Crear la entidad JPA en infrastructure/persistence/jpa/{Entidad}Entity.java.
6. Crear el repositorio en infrastructure/persistence/jpa/{Entidad}JpaRepository.java.
7. Crear el mapper en infrastructure/mapper/{Entidad}Mapper.java con las 4 conversiones.
8. Crear los DTOs en infrastructure/controller/dto/{Entidad}Request.java y {Entidad}Response.java.
9. Crear el caso de uso en application/usecase/{Entidad}UseCase.java con los 6 métodos estándar.
10. Crear el controller en infrastructure/controller/{Entidad}Controller.java.
11. Verificar que el AuditoriaMongoAspect captura la nueva entidad (automático).
12. Si hay archivos, inyectar StoragePort en el caso de uso.
13. Documentar con el estándar de comentarios: cabecera, @author, @version, @param, @return.
14. Crear las pruebas unitarias en test/usecase/{Entidad}UseCaseTest.java y test/mapper/{Entidad}MapperTest.java siguiendo la sección 20.

19.4 Reglas de Oro

- Nunca borrar físicamente un registro. Siempre usar status=E con deleted_by y deleted_at.
- Nunca exponer entidades JPA fuera del paquete persistence/jpa.
- Nunca poner lógica de negocio en los controllers.
- Nunca poner credenciales en el código fuente ni en application.yaml. Usar Vault o .env.
- Siempre documentar con el estándar de comentarios antes de hacer commit.
- Siempre agregar los 7 campos de auditoría a toda nueva tabla del sistema.

20. Pruebas

20.1 Estructura de Paquetes de Prueba

Los archivos de prueba se organizan en el directorio `src/test/java/` replicando parcialmente la estructura del código fuente, pero bajo subpaquetes específicos por tipo de prueba:

Elemento	Descripción
Ubicación raíz	<code>src/test/java/{groupId}.{artifactId}/</code>
Clase de integración	<code>{Application}Tests.java</code> — prueba de contexto Spring Boot
Subpaquete usecase	<code>test/usecase/{Entidad}UseCaseTest.java</code>
Subpaquete mapper	<code>test/mapper/{Entidad}MapperTest.java</code>
Subpaquete adicional	<code>test/{capa}/{Clase}Test.java</code> — para otras capas según necesidad

20.2 Tecnologías y Dependencias

Herramienta	Versión	Propósito
JUnit 5 (jupiter)	5.x (provista por Spring Boot Starter Test)	Framework de pruebas unitarias y de integración.
Mockito	5.x (provista por Spring Boot Starter Test)	Mocking de dependencias para aislar la lógica bajo prueba.
AssertJ	3.x (provista por Spring Boot Starter Test)	Aserciones fluidas con <code>assertThat()</code> y <code>assertThatThrownBy()</code> .
Spring Boot Test	3.3.5	Prueba de integración con <code>@SpringBootTest</code> para verificar el contexto.
MockMultipartFile	Spring Test	Simulación de archivos multipart para pruebas de carga de documentos.

20.3 Patrones de Prueba por Capa

20.3.1 Pruebas de Casos de Uso (UseCase)

Verifican la lógica de negocio en aislamiento. Usan `@ExtendWith(MockitoExtension.class)` para inyectar mocks de las dependencias (repositorios JPA, mappers y storage).

Estructura obligatoria:

```
@ExtendWith(MockitoExtension.class) @DisplayName("/{Entidad}UseCase - Pruebas Unitarias") class  
{Entidad}UseCaseTest {
```

```
@Mock  
private {Entidad}JpaRepository repository;
```

```
@Mock  
private {Entidad}Mapper mapper;
```

```
@InjectMocks
private {Entidad}UseCase useCase;
```

```
private {Entidad} entidadDominio;
```

```
private {Entidad}Entity entidadEntity;
```

```
private UUID id;
```

```
@BeforeEach
```

```
void setUp() {
    id = UUID.randomUUID();
    entidadDominio = {Entidad}.builder()
        .id(id)
        .// campos de negocio
        .status("A")
        .build();

    entidadEntity = {Entidad}Entity.builder()
        .id(id)
        .// campos de negocio
        .status("A")
        .build();
}
```

```
// — MÉTODO —
```

```
@Test
```

```
@DisplayName("metodo: descripcion del escenario exitoso")
```

```
void metodo_exitoso() {
    when(repository.{metodo}(...)).thenReturn(...);
    when(mapper.toDomain(...)).thenReturn(entidadDominio);
```

```
var resultado = useCase.{metodo}(...);
```

```
assertThat(resultado).isNotNull();
assertThat(resultado.getCampo()).isEqualTo("valor");
verify(repository, times(1)).{metodo}(...);
}
```

```
@Test
```

```
@DisplayName("metodo: lanza excepcion cuando la entidad no existe")
```

```
void metodo_noExiste_lanzaExcepcion() {
    when(repository.{metodo}(...)).thenReturn(Optional.empty());
```

```
assertThatThrownBy(() -> useCase.{metodo}(...))
    .isInstanceOf(IllegalArgumentException.class)
    .hasMessageContaining("mensaje esperado");
```

```
verify(repository, never()).save(any());
}
```

Reglas:

- Cada método del UseCase debe tener al menos un escenario exitoso y uno de error.
- Usar verify() para validar que las interacciones con repositorios ocurren exactamente una vez.
- Usar verify(..., never()) para validar que NO se ejecutan operaciones no deseadas.
- Usar argThat() para validar argumentos específicos en métodos mockeados.
- Los datos de prueba se inicializan en @BeforeEach para evitar duplicación.

20.3.2 Pruebas de Mappers (Mapper)

Verifican que las conversiones entre Entity, Domain, Request y Response mapean correctamente todos los campos. No requieren Mockito; el mapper se instancia directamente.

Estructura obligatoria:

```
@DisplayName("{Entidad}Mapper - Pruebas Unitarias") class {Entidad}MapperTest {
```

```
    private {Entidad}Mapper mapper;  
    private UUID id;  
    private LocalDateTime ahora;
```

```
    @BeforeEach  
    void setUp() {  
        mapper = new {Entidad}Mapper();  
        id = UUID.randomUUID();  
        ahora = LocalDateTime.now();  
    }
```

```
    // — toDomain(Entity) —————
```

```
    @Test  
    @DisplayName("toDomain(Entity): mapea correctamente todos los campos")  
    void toDomain_desdeEntity_mapeaCorrectamente() { ... }
```

```
    @Test  
    @DisplayName("toDomain(Entity): retorna null cuando entity es null")  
    void toDomain_desdeEntity_retornaNull_cuandoEntityEsNull() {  
        assertThat(mapper.toDomain(({Entidad}Entity) null)).isNull();  
    }
```

```
    // — toEntity(Domain) —————
```

```
    @Test  
    @DisplayName("toEntity(Domain): mapea correctamente todos los campos")  
    void toEntity_desdeDomain_mapeaCorrectamente() { ... }
```

```
    @Test  
    @DisplayName("toEntity(Domain): asigna status A cuando el dominio no tiene status")  
    void toEntity_desdeDomain_asignaStatusA_cuandoStatusEsNull() { ... }
```

```
    @Test  
    @DisplayName("toEntity(Domain): retorna null cuando domain es null")  
    void toEntity_desdeDomain_retornaNull_cuandoDomainEsNull() { ... }
```

```
    // — toDomain(Request) —————
```

```
    @Test  
    @DisplayName("toDomain(Request): mapea correctamente los campos de negocio")  
    void toDomain_desdeRequest_mapeaCorrectamente() { ... }
```

```
    @Test  
    @DisplayName("toDomain(Request): no incluye id ni campos de auditoria")  
    void toDomain_desdeRequest_noIncluyeAuditoria() { ... }
```

```
    @Test  
    @DisplayName("toDomain(Request): retorna null cuando request es null")  
    void toDomain_desdeRequest_retornaNull_cuandoRequestEsNull() { ... }
```

```
    // — toResponse(Domain) —————
```

```

@Test
@DisplayName("toResponse(Domain): mapea correctamente todos los campos incluyendo auditoria")
void toResponse_desdeDomain_mapeaCorrectamente() { ... }

@Test
@DisplayName("toResponse(Domain): retorna null cuando domain es null")
void toResponse_desdeDomain_retornaNull_cuandoDomainEsNull() { ... }
}

```

Método	Entrada	Salida
toDomain(Entity)	{Entidad}Entity	{Entidad}
toEntity(Domain)	{Entidad}	{Entidad}Entity
toDomain(Request)	{Entidad}Request	{Entidad}
toResponse(Domain)	{Entidad}	{Entidad}Response

Reglas:

- Cada método de conversión debe probar: mapeo correcto + null input.
- toEntity(Domain) debe verificar que asigna status="A" por defecto.
- toDomain(Request) debe verificar que NO incluye id ni campos de auditoría.
- toResponse(Domain) debe verificar que incluye los 7 campos de auditoría.

20.3.3 Prueba de Integración

Una única clase que verifica que el contexto de Spring Boot se levanta correctamente:

```

@SpringBootTest class {Application}Tests {

    @Test
    void contextLoads() {
    }
}

```

Esta prueba se encuentra en el paquete raíz de pruebas (src/test/java/{groupId}.{artifactId}/) y garantiza que todas las configuraciones, beans y dependencias se resuelven sin errores al arrancar.

20.4 Convenciones de Nomenclatura

Elemento	Convención	Ejemplo
Clase de prueba	{ClaseBajoPrueba}Test	{Entidad}UseCaseTest, {Entidad}MapperTest
Método de prueba	{metodo}_{escenario}	{metodo}_{escenario} ej: crear_duplicado_lanzaExcepcion
DisplayName	"metodo: descripcion del escenario"	"{metodo}: exitoso cuando los datos son válidos"

Elemento	Convención	Ejemplo
Separación visual	// — MÉTODO — con guiones	//—CREAR—————
Nombre de variables mock	Mismo nombre que el campo en la clase real	repository, mapper, storagePort
Constantes de prueba	Inicializadas en @BeforeEach	idPersona, personaDominio, personaEntity

20.5 Cobertura Esperada por Método

20.5.1 Casos de Uso

Método del UseCase	Escenario exitoso	Escenarios de error obligatorios
crear()	Persiste y retorna la entidad con valores por defecto	Dato único duplicado (por cada campo único de la entidad)
actualizar()	Actualiza campos y retorna la entidad	Entidad no encontrada
cambiarStatus()	Cambia de un estado válido a otro	Estado inválido o transición no permitida
eliminar()	Asigna status de eliminación y datos de auditoría	Entidad no encontrada
buscarPorId()	Retorna la entidad activa	Entidad no encontrada
buscarPor{CampoUnico}()	Retorna la entidad por campo único identificador	Valor único no encontrado
listarActivos()	Retorna página con registros activos	Página vacía sin registros
{metodoConDependencia}()	Ejecuta operación y persiste metadata	Dependencia no encontrada

20.5.2 Mappers

Método del mapper	Escenarios obligatorios
toDomain(Entity)	Mapeo correcto de todos los campos, null input
toEntity(Domain)	Mapeo correcto, valores por defecto del proyecto, null input
toDomain(Request)	Mapeo correcto, exclusión de campos internos (id/auditoría), null input
toResonse(Domain)	Mapeo correcto incluyendo campos de auditoría, null input

20.6 Checklist para Agregar Pruebas de Nueva Entidad

Al agregar una nueva entidad al proyecto, deben crearse los siguientes archivos de prueba:

- Crear test/usecase/{Entidad}UseCaseTest.java con los escenarios de la tabla 20.5.1.
- Crear test/mapper/{Entidad}MapperTest.java con los escenarios de la tabla 20.5.2.
- Crear test/{capa}/{Clase}Test.java por cada capa adicional que requiera pruebas.
- Verificar que cada método del UseCase pruebe al menos un escenario exitoso y uno de error.
- Verificar que cada método de conversión del mapper pruebe mapeo correcto y null input.
- Verificar que los mocks validen interacciones con dependencias (existencia y cantidad de llamadas).
- Ejecutar el comando de pruebas del proyecto (ej: mvn test, gradle test) y confirmar que todas pasan.
- Verificar que cada clase de prueba incluya la cabecera de copyright del proyecto y Javadoc con descripción.

21. Anexos

21.1 Anexo 1 — .env.example

Plantilla de variables de entorno que debe versionarse en Git. Los desarrolladores la copian como .env y completan los valores:

```
# IP_SERVER= (vacío=local | IP=dockerizado, ej: 192.168.12.42)
IP_SERVER=
SERVER_PORT=8080
SERVER_CONTEXT_PATH=/api
DB_ENGINE=postgres

# PostgreSQL [VAULT]
DB_POSTGRES_HOST=
DB_POSTGRES_PORT=
DB_POSTGRES_NAME=
DB_POSTGRES_USERNAME=
DB_POSTGRES_PASSWORD=
DB_POSTGRES_DDL_AUTO=update
DB_POSTGRES_SHOW_SQL=false

# Vault
VAULT_ENABLED=false
VAULT_HOST=localhost
VAULT_PORT=8200
VAULT_SCHEME=http
VAULT_TOKEN=

# MongoDB [VAULT]
MONGO_ENABLED=false
DB_MONGO_HOST=
DB_MONGO_PORT=
DB_MONGO_NAME=

# MinIO [VAULT]
MINIO_ENABLED=false
```



```

MINIO_URL=
MINIO_ACCESS_KEY=
MINIO_SECRET_KEY=
MINIO_BUCKET_DEFAULT=base-spring-bucket

# RSA [VAULT]
RSA_ENABLED=false
RSA_PUBLIC_KEY=
RSA_PRIVATE_KEY=

```

21.2 Anexo 2 — DDL Genérico con Campos de Auditoría

Estructura mínima que debe tener toda tabla del sistema en PostgreSQL:

```

CREATE TABLE {nombre_tabla} (
    id                UUID                NOT NULL DEFAULT gen_random_uuid(),
    -- Campos de negocio propios de la entidad
    status            CHAR(1)            NOT NULL DEFAULT 'A',
    created_by        VARCHAR(100),
    created_at        TIMESTAMP(6) NOT NULL,
    updated_by        VARCHAR(100),
    updated_at        TIMESTAMP(6),
    deleted_by        VARCHAR(100),
    deleted_at        TIMESTAMP(6),
    CONSTRAINT pk_{nombre_tabla} PRIMARY KEY (id)
);

CREATE INDEX idx_{nombre_tabla}_status ON {nombre_tabla} (status);

```

21.3 Anexo 3 — Log de Arranque

Al levantar la aplicación se muestra el siguiente resumen en el log:

```

-----
BaseSpringApi - IESS | Arranque completado
-----

BD Relacional : POSTGRES | Habilitado/Interno | localhost:5432/base_spring_db
MongoDB       : Habilitado/Interno | localhost:27017/AUDITORIA_IESS
Vault         : Habilitado/Interno | localhost:8200/BaseSpringApi/
MinIO         : Habilitado/Interno | http://localhost:9000 | Bucket: base-
spring-bucket
-----

Swagger UI    : http://localhost:8080/api/swagger-ui.html
-----

```